Toronto Metropolitan University

Department of Electrical, Computer & Biomedical Engineering
Faculty of Engineering & Architectural Science

Program: Computer Engineering

Course Number	COE 892
Course Title	Distributed Cloud Computing
Semester/Year	Winter 2026
Instructor	Muhammad Jaseemuddin

Report Title	Interim Report
GitHub Link	Git

Section No.	09
Submission Date	March 8, 2026
Due Date	March 9, 2026

Name	Student ID	Signature*
Krish Patel	xxxx91722	K.P

(Note: remove the first 4 digits from your student ID)

*By signing above you attest that you have contributed to this submission and confirm that all work you have contributed to this submission is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: https://www.torontomu.ca/content/dam/senate/policies/pol60.pdf

Abstract (Draft)

Small ProxMox-based clusters are useful for teaching virtualization, distributed systems, and service orchestration, but in practice they are often operated through manual and inconsistent workflows. In many lab environments, tasks are still launched by logging into a specific virtual machine, running scripts manually, checking outputs in different directories, and recovering from faults only after noticing that a worker or service has stopped responding. This makes demonstrations harder to repeat and makes system behavior difficult to defend during evaluation. ProxSyncQ is a lightweight fault-aware execution framework for a small ProxMox cluster that addresses this problem by combining shared storage, worker daemons, lease-based reclaim, and a Raspberry Pi-hosted operations dashboard.

The current ProxSyncQ deployment consists of three worker VMs and one Raspberry Pi coordinator on the lab network. The Raspberry Pi runs at 10.26.0.170, while the worker VMs run at 10.26.0.171, 10.26.0.172, and 10.26.0.173. Shared storage is mounted consistently across the workers at /srv/proxsyncq/shared, which acts as the durable location for queue records, logs, and artifacts. Worker daemons on the VMs poll for jobs, claim work, execute handlers, and update queue state. A Raspberry Pi-hosted dashboard provides centralized visibility into node status, shared storage health, queue counts, and recent job history. A controlled lifecycle demo already shows that a job can be submitted, leased by one worker, interrupted mid-execution, returned to pending after lease expiry, reclaimed by another worker, and then completed successfully.

At the current interim stage, the project has moved beyond planning and into a working prototype. Static addressing, peer connectivity, shared storage visibility, worker daemon execution, dashboard monitoring, and a full reclaim demo are already operational. The ProxMox VE API and a dedicated web API are not yet integrated, so current recovery is demonstrated through controlled worker failure rather than automated VM recovery. The final stage of the project will focus on hardening the queue path, formalizing the job schema, improving event history and dashboard visibility, and adding safe VM-level recovery through the ProxMox API.

Introduction

ProxMox-based clusters are commonly used in teaching labs because they allow multiple virtual machines, controlled network layouts, and repeatable distributed systems experiments to be built on a single physical environment. Despite these advantages, small lab clusters are often still run in a very manual way. A user may pick one machine to run a task, wait for it to finish, and then manually investigate failures if that node crashes or becomes unhealthy. This approach creates inconsistency between test runs, weakens reproducibility, and makes it harder to explain exactly what happened during a demonstration.

ProxSyncQ is designed to make this environment more structured and fault aware. Instead of treating each VM as an isolated execution point, the project turns the cluster into a shared execution pool backed by durable shared storage. Jobs are submitted once, represented as persistent records, claimed by a healthy worker, and tracked through visible state transitions. If a worker becomes unavailable during execution, the job is not lost. It remains visible on shared storage and can be reclaimed by another worker after lease expiry. This approach makes the cluster easier to reason about because execution and recovery can be observed directly rather than inferred from scattered logs.

The current implementation is tied to a real lab setup and not just a conceptual design. The system currently operates on **four active nodes**: the Raspberry Pi coordinator at **10.26.0.170** and three worker VMs at **10.26.0.171**, **10.26.0.172**, and **10.26.0.173**. Shared storage is available at **/srv/proxsyncq/shared**, and the queue lifecycle uses the visible states **pending**, **leased**, **running**, **completed**, and **failed**. The Raspberry Pi hosts the operations dashboard and runs a live lifecycle demo server. In the recorded demo, a job entered pending at **23:53:05**, was leased by VM2 at **23:53:10**, began running on VM2 at **23:53:14**, was deliberately interrupted at **23:53:20**, returned to pending after lease expiry at **23:53:29**, was leased by VM1 at **23:53:33**, and completed successfully on VM1 at **23:53:45**. That run demonstrates that the current system can already show real-time reclaim behavior across workers.

ProxSyncQ does not currently aim to solve large-scale scheduling or provide strict exactly once execution under every possible failure condition. Instead, it targets **durable at-least-once execution**, visible reclaim behavior, and strong auditability. That design goal matches the needs of a course project because it prioritizes correctness, observability, and recovery in a modest cluster where clarity of behavior matters more than large-scale throughput.

System Context and Design Targets

The current design is intentionally sized for the existing lab environment: **three worker VMs** and **one out-of-band Raspberry Pi**. This keeps the system small enough to observe directly while still supporting realistic distributed behavior such as worker failure, reclaim, shared storage coordination, and node-level monitoring. The queue is built around a shared filesystem rather than a separate distributed coordinator. In this context, that is a deliberate choice because shared filesystem state is easy to inspect and aligns with the cluster's existing Gluster-backed shared storage setup.

The project is guided by a few measurable design targets. First, every submitted job should reach a terminal state, either **completed** or **failed**, so that the system can satisfy the accounting condition:

submitted = completed + failed

Second, reclaim after worker loss should be bounded and explainable. Based on the recorded demo, the reclaim path is already visible and measurable. In the observed run, VM2 was stopped at **23:53:20**, the job returned to pending at **23:53:29**, and VM1 reclaimed it at **23:53:33**. That gives a measured failure-to-requeue delay of about **9 seconds** and a failure-to-reclaim delay of about **13 seconds** for the current demo configuration. Since no formal runtime policy has been finalized yet, these values are best treated as observed prototype timings rather than final guarantees.

A third design target is end-to-end traceability. For each job, the system records timestamps that allow queue timing to be analyzed. The key metrics are:

- **Queue delay = t_lease − t_submit**
- **Execution time = t_completed − t_start**
- **End-to-end latency = t_completed − t_submit**

From the recorded lifecycle run:

- job submission: **23:53:05**
- first lease: **23:53:10**
- first execution start: **23:53:14**
- reclaim to pending: **23:53:29**
- second lease: **23:53:33**
- second execution start: **23:53:37**
- completion: **23:53:45**

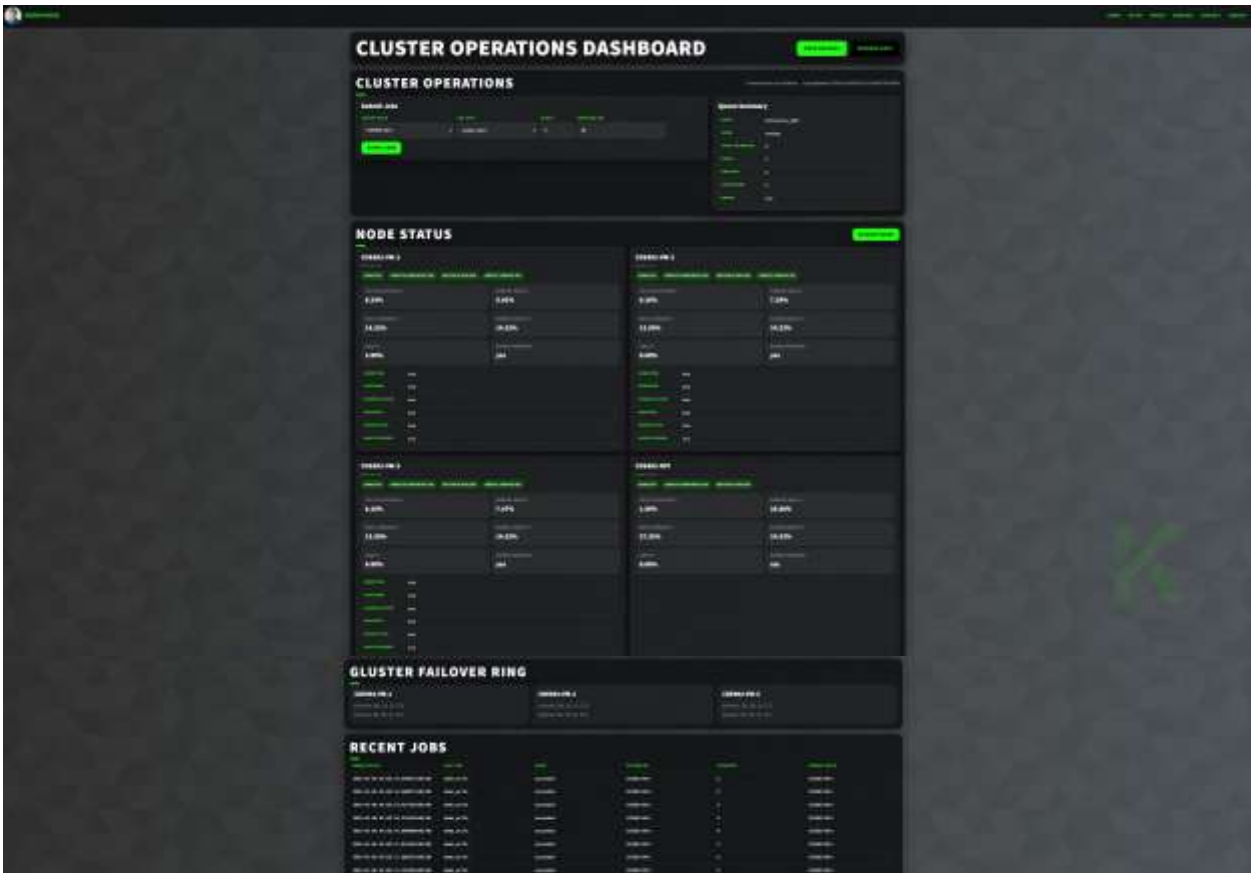
These values show that the prototype is already generating useful quantitative evidence rather than only qualitative descriptions.

System Design

ProxSyncQ currently has four main parts: shared storage, worker daemons on the VMs, a Raspberry Pi-hosted dashboard and demo controller, and a durable file-based queue state model. Shared storage holds job records, state changes, logs, and artifacts. The worker daemons on the three VMs poll for jobs, claim work, execute it, and update the shared state. The Raspberry Pi serves as the main visibility point by hosting the operations dashboard and lifecycle demo. ProxMox API recovery is planned, but it is not yet active.

A key design choice is that there is no permanent master worker. Instead, coordination happens through shared storage and visible queue files, which makes the system easier to inspect during testing and demonstration. Each job is stored as a structured record containing its ID, type, payload, attempt count, lease details, and log or artifact references. The lifecycle uses five visible states: pending, leased, running, completed, and failed. This state model is already shown in the live demo and dashboard views.

Observability is already part of the working system. The Raspberry Pi-hosted dashboard shows node status, queue summary, Gluster failover information, and recent jobs. The node cards also report resource and health data such as CPU usage, memory usage, root storage usage, shared mount presence, and service checks.



Web layout for the Operation Dashboard

The dashboard also includes service readiness checks that report components such as RabbitMQ and PostgreSQL as healthy on the VMs. However, based on the current implementation, these services should be described as **monitored infrastructure components** rather than active dependencies of the queue path. The present lifecycle demo and reclaim behavior are already functioning through shared storage and worker daemons, without requiring the ProxMox API or a separate web API. This distinction is important because it keeps the report aligned with what is already working.

Security and control are currently handled conservatively. The Raspberry Pi is the main control and visibility point, and worker coordination happens inside the lab network. ProxMox API control is planned for later stages, but it is not yet integrated into the active recovery path. At this stage, the system demonstrates safe worker-level reclaim rather than automated VM-level restart.

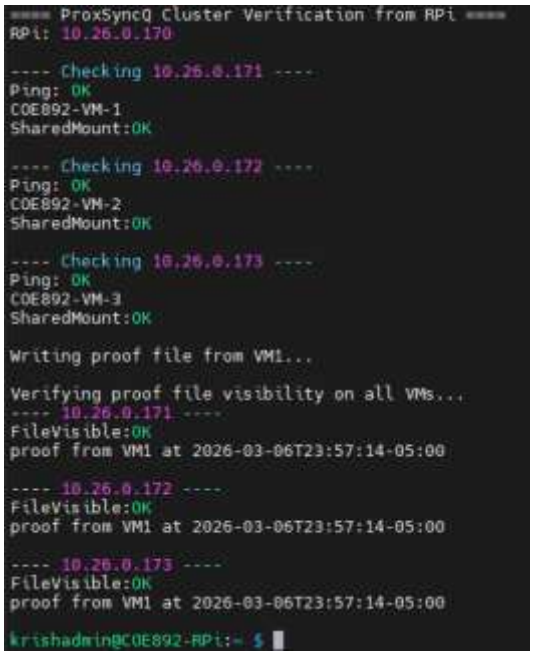
Failure handling is based on leases. If a worker crashes while holding a job, the job remains on shared storage and can later be reclaimed by another healthy worker after lease expiry. If a full VM becomes unhealthy, the Raspberry Pi can detect that and trigger a controlled recovery action through the ProxMox VE API. Because the Pi is out of band, it can still coordinate recovery even when a worker VM is unreachable.

Observability is a core requirement. The Raspberry Pi hosts a dashboard showing worker health, shared mount status, queue depth, and recent job history. At minimum, it should report counts for pending, leased, running, done, and failed jobs. This allows an evaluator to understand system state quickly without manually checking every node.

Security is handled through least privilege and auditability. ProxMox API access will use limited tokens, and every automated recovery action will be logged with its timestamp, target, and reason.

Partial Implementation and Current Progress

The project has completed its core infrastructure baseline and already demonstrates meaningful distributed behavior. The lab network uses stable static IPs, with the Raspberry Pi at **10.26.0.170** and worker VMs at **10.26.0.171**, **10.26.0.172**, and **10.26.0.173**. Shared storage is mounted consistently across the worker VMs at **/srv/proxsyncq/shared**, and cross-node visibility has been verified. Worker daemons are running on the VMs, and the Raspberry Pi is already serving the dashboard and lifecycle demo interface. This means the system has moved beyond a design-only stage and now has a functioning execution and monitoring baseline.



Infrastructure view from RPI's standpoint

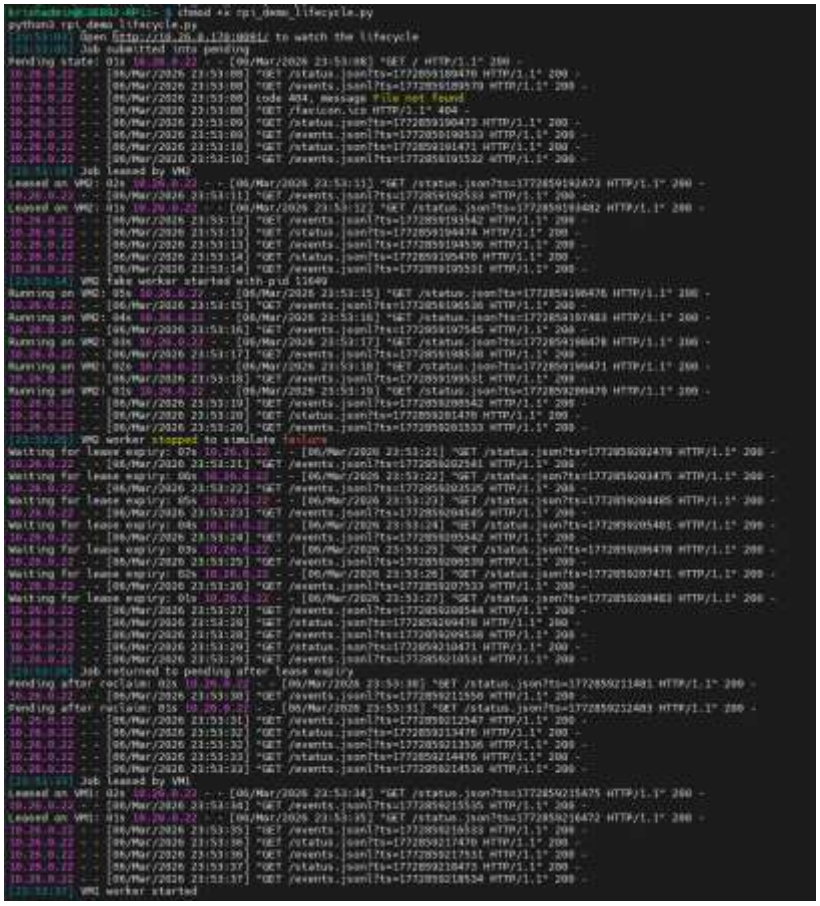
Evaluation Plan

The evaluation will focus on correctness, recovery behavior, observability, and coordination overhead, with several of these already partially demonstrated by the current prototype. The cluster currently has 4 active nodes, and the Raspberry Pi already provides centralized visibility into node state, queue summary, and recent job history. Shared storage is active on the workers, and the system has already demonstrated that a job can survive a worker failure and still complete on another VM. This means the evaluation is no longer starting from a theoretical baseline, but from an implementation that already produces measurable events and timings.

Correctness will be evaluated by checking that every submitted job reaches a terminal state, so that the system satisfies:

submitted = completed + failed

The reclaim demo already supports this direction. In the recorded run, a single job was submitted once and completed once, even though one worker was interrupted mid-execution. Recovery behavior will be evaluated through controlled failure tests similar to the current demo. Based on the observed log, the worker was stopped at 23:53:20, the job returned to pending at 23:53:29, and the new worker reclaimed it at 23:53:33. These values provide an initial measured reclaim sequence of about 9 seconds to requeue and 13 seconds to reclaim in the current prototype. Future evaluation runs can compare these timings across repeated tests.



Demo of the Job Lifecycle

Observability will be evaluated through the Raspberry Pi dashboard and the lifecycle event stream. The dashboard already exposes node-level health data, queue state, Gluster failover information, and recent completed jobs. The lifecycle demo also provides real-time status updates through the Raspberry Pi at port 8091. An evaluator can therefore watch the queue state change live and verify the event order against timestamps, which makes the system much easier to defend during demonstration. Performance evaluation will use the recorded timestamps for submission, leasing, running, reclaim, and completion. Since the queue path currently operates across three worker VMs, throughput and delay can be measured under fixed workloads, while overhead can be estimated by comparing active execution time with total end-to-end time.

Timeline and Risks

Because the baseline cluster, worker daemons, shared storage, and dashboard are already working, the remaining work is now more focused than it was in the original plan. The next phase is to move from a controlled demo path into a cleaner main queue implementation with a stable directory layout, finalized job schema, and better event history management. After that, reclaim behavior should be exercised under repeated tests rather than a single scripted demo. Once the queue path is stable, the Raspberry Pi dashboard can be extended from health monitoring into a fuller operational interface that shows current job ownership, richer history, and clearer reclaim reasoning. After those steps, the ProxMox VE API can be integrated as the out-of-band VM recovery layer.

The risks are now clearer because implementation has already exposed what works and what still needs attention. Basic connectivity is no longer the main risk, since node communication, worker execution, and shared storage are already functioning. The larger technical risk is now correct coordination during lease expiry and reclaim, especially if multiple workers compete for a returned job at the same time. This will be mitigated by keeping the queue state model simple and making every transition explicit in durable logs. Another realistic risk is duplicate execution, since the current design is at-least-once rather than exactly-once. That risk will be reduced by careful lease handling and by designing job handlers, so they are idempotent where possible. A third important risk is premature complexity. The dashboard is already useful, so the project should avoid over-polishing the UI at the expense of queue hardening. Finally, ProxMox API recovery is still not working, so any report language about VM-level restart must remain clearly in the future-work category until it has been integrated and tested.

References (Draft)

[1] FastAPI, “Background Tasks,” *FastAPI Documentation*. [Online]. Available: https://fastapi.tiangolo.com/tutorial/background-tasks/

[2] Grafana Labs, “About Grafana,” *Grafana Documentation*. [Online]. Available: https://grafana.com/docs/grafana/latest/introduction/

[3] Gluster Docs, “Administration Guide,” *Gluster Docs*. [Online]. Available: https://docs.gluster.org/en/main/Administrator-Guide/

[4] Gluster Docs, “Setting up GlusterFS Volumes,” *Gluster Docs*. [Online]. Available: https://docs.gluster.org/en/main/Administrator-Guide/Setting-Up-Volumes/

[5] M. Kerrisk, “open(2) - Linux manual page,” *man7.org*. [Online]. Available: https://man7.org/linux/man-pages/man2/open.2.html

[6] M. Kerrisk, “rename(2) - Linux manual page,” *man7.org*. [Online]. Available: https://man7.org/linux/man-pages/man2/rename.2.html

[7] PostgreSQL Global Development Group, “JSON Types,” *PostgreSQL Documentation*. [Online]. Available: https://www.postgresql.org/docs/current/datatype-json.html

[8] PostgreSQL Global Development Group, “Transactions and Locking,” *PostgreSQL Documentation*. [Online]. Available: https://www.postgresql.org/docs/current/xact-locking.html

[9] Prometheus Authors, “Instrumentation,” *Prometheus Documentation*. [Online]. Available: https://prometheus.io/docs/practices/instrumentation/

[10] Prometheus Authors, “Storage,” *Prometheus Documentation*. [Online]. Available: https://prometheus.io/docs/prometheus/latest/storage/

[11] Proxmox Server Solutions GmbH, “Proxmox VE API,” *Proxmox VE Wiki*. [Online]. Available: https://pve.proxmox.com/wiki/Proxmox_VE_API

[12] Proxmox Server Solutions GmbH, “Proxmox VE API Documentation,” *Proxmox VE API Viewer*. [Online]. Available: https://pve.proxmox.com/pve-docs/api-viewer/

[13] RabbitMQ Team, “Exchanges,” *RabbitMQ Documentation*. [Online]. Available: https://www.rabbitmq.com/docs/exchanges

[14] RabbitMQ Team, “Publishers,” *RabbitMQ Documentation*. [Online]. Available: https://www.rabbitmq.com/docs/publishers

[15] RabbitMQ Team, “Work Queues,” *RabbitMQ Tutorials*. [Online]. Available: https://www.rabbitmq.com/tutorials/tutorial-two-python